

RAG 文档问答系统 — RAGAS 量化评测报告

项目: Telegram Clone — AI 智能助手 | 代码: github.com/ttb-eng/-telegram-clone | 线上: mytelegram.xyz/docs

一、项目概述

在 Telegram Clone 项目中实现了基于 RAG（检索增强生成）的文档问答功能。用户上传文档后，系统自动分块、向量化并存入 ChromaDB 向量数据库；提问时检索相关片段，交由 DeepSeek 大模型增强生成回答。为量化评估系统质量，基于 RAGAS 框架设计了 15 题跨文档评测方案，覆盖四项核心指标。

二、技术架构

文档入库: chunk_text (500 字符滑动窗口) → SentenceTransformer (all-MiniLM-L6-v2, 384 维) → ChromaDB (cosine 空间, 用户级隔离)

问答检索: 用户提问 → Embedding → Cosine 检索 (top-5 + distance<0.8 过滤) → 拼接上下文 → DeepSeek Chat → 增强生成回答

组件	技术选型
向量数据库	ChromaDB (PersistentClient)
Embedding 模型	sentence-transformers / all-MiniLM-L6-v2 (384 维)
LLM	DeepSeek Chat API
评测框架	RAGAS
用户隔离	user_id 元数据过滤 + 闭包注入

三、评测方案

评测文档（3 篇，跨域设计）：python_decorator.txt（Python 装饰器） | redis_cache.txt（Redis 缓存策略） | search_engine.txt（搜索引擎原理）

题型	示例	数量
单文档提取	"装饰器使用什么符号？"	6 题
跨文档对比	"倒排索引和装饰器都用到了什么共同概念？"	4 题
多文档整合	"三篇文档中分别提到了哪些排序相关内容？"	5 题

四、评测结果

指标	分数	说明
Faithfulness（忠实度）	0.73	生成回答与检索内容的吻合程度
Answer Relevancy（回答相关性）	0.77	回答与问题的语义相关性
Context Recall（上下文召回）	0.79	检索到的内容覆盖标准答案的比例
Context Precision（上下文精度）	0.36	检索结果中相关内容的比例（已定位优化方向）

评测环境：15 题 × 4 指标 = 60 个并行评估任务，评测 LLM 为 DeepSeek Chat，Embedding 为 all-MiniLM-L6-v2，完整可复现（tests/test_rag_ragas.py）。

五、优化方向

已完成：引入 distance 阈值过滤（cosine<0.8）；自研 DeepSeekJsonWrapper 解决 DeepSeek 返回 JSON 带 Markdown 围栏的兼容问题

Chunking 策略：当前 chunk_size=500，过粗切片引入无关内容，计划调整为 200~300 并增大 overlap

Re-rank：检索 top-5 后引入 cross-encoder 重排序，提升 context_precision

混合检索：BM25 + 向量检索融合，提升中英文混合场景召回率

六、技术亮点

- 全异步架构：FastAPI + SQLAlchemy Async + httpx，检索和 LLM 调用均为异步
- 用户级知识库隔离：ChromaDB metadata 按 user_id 过滤，search_knowledge 通过闭包注入 user_id
- 评测可复现：15 组 eval cases 写死在测试文件，pytest 一键运行
- DeepSeek 兼容适配：基于 LangChain BaseChatModel 自定义封装，解决 JSON 输出带 Markdown 围栏的兼容问题
- 30+ API 已上线：Nginx 反代部署至阿里云 ECS，Swagger 文档可在线调试